

Week 8 Task Document — What / Why / Where

Agent Failure Modes & Trajectory Evals. Repository [dineshrprofessional-design/Gen-AI-Dev](#), branch [ragew8/trajectory-evals](#), commit [e71c8f2](#). Dated 2026-09-22.

Every Week 8 requirement below is answered in three parts: **what** was built, **why** it was built that way, and **where** the code is — file and line, so a reviewer can go straight to it.

Task index

#	Task	Where	Status
1	Trajectory test suite	eval/week8_cases.yaml	done
2	Telemetry / metrics engine	app/rag/trajectory_eval.py	done
3	Outcome-vs-trajectory gap	report/w8/gap_analysis.txt	done
4	Exactly one mitigation	app/rag/week8_mitigation.py	done
5	Regression matrix, six modes	report/w8/regression_matrix.md	done
6	Bonus defences	app/rag/week8_defense.py	done
7	Machine telemetry	report/w8/trajectory_eval.json	done

Task 1 — Trajectory test suite

What we did

Authored 10 trajectory cases, one per Week 7 documentation query, each declaring the tool paths that count as correct, the argument schema each tool must satisfy, and the step bounds.

Why we did it

Week 7 scored only the final answer. An agent can reach a right answer down a wrong path, or a right path and fumble the wording — and you cannot tell which without inspecting the path itself.

Three design decisions and their reasons:

Flexible paths, not a fixed expected sequence. An expected-sequence check fails an agent that reached the same place by an equally valid route. That would make "tool-choice accuracy" a measure of *conformity*, not correctness. Instead a trajectory passes a conjunction of constraints — any shape satisfying all of them is correct.

Commutative ordering where order carries no meaning. [w8-04](#) and [w8-05](#) compare a default across v2 and v3. Both calls are independent; neither informs the other. Penalising v3-before-v2 would be scoring an arbitrary choice.

Ordering enforced only where a dependency is real. The four two-hop cases need [check_deprecation](#) before [search_docs](#), because step 2's [api_version](#) cannot exist until step 1 returns the replacement pointer. Searching first means the version was guessed.

Where the code is

Thing	Location
The spec file	<code>eval/week8_cases.yaml</code> (278 lines)
Case list begins	<code>eval/week8_cases.yaml:47</code>
First case <code>w8-01</code>	<code>eval/week8_cases.yaml:49-70</code>
<code>forbidden_tools</code> example	<code>eval/week8_cases.yaml:57</code>
<code>ordering / commutative</code> fields	<code>eval/week8_cases.yaml:58-59</code>
Schema parsed into	<code>TrajectoryCase, app/rag/trajectory_eval.py:75</code>
Argument schema parsed into	<code>ArgSpec, app/rag/trajectory_eval.py:68</code>
Loader	<code>load_cases(), app/rag/trajectory_eval.py:93</code>
Spec validator	<code>validate(), app/rag/trajectory_eval.py:526</code>

How to check it

```
python -m app.rag.trajectory_eval --validate
```

Fails if any referenced tool, SDK method or path does not exist in the corpus; if a case is both required and forbidden; if `min_steps > max_steps`; if a rationale is missing; or if `ordering` is set on a commutative case.

Task 2 — Telemetry / metrics engine

What we did

An engine that reads saved `AgentRun` JSON dumps and computes five metrics: Tool-Choice Accuracy, Argument Validity Rate, Step Efficiency, Cost p50 and Cost Max.

Why we did it

Replay, not re-run. Everything reads a saved dump — no model, no network, no tokens. The same JSON scores identically on any machine, which is what makes a before/after comparison a *measurement* rather than two separate anecdotes taken at different times.

Tool choice and argument validity are kept separate. The right tool with a hallucinated path is a different failure from the wrong tool. Collapsing them into one number would make both unreadable.

Both p50 and Max are reported. A thrashing agent has an unremarkable median and a catastrophic tail. Reporting only p50 hides exactly the failure mode this week targets — the baseline proves it: p50 cost \$0.000810, Max \$0.003246, a 4x spread.

Step Efficiency is capped at 1.0 so a case finishing in fewer than `min_steps` cannot score above perfect and mask an inefficiency elsewhere.

Where the code is

Metric	Formula location
Tool-Choice Accuracy	<code>trajectory_eval.py:377 — correct_tool / total</code>
Argument Validity	<code>trajectory_eval.py:379 — valid_args / total</code>
Step Efficiency p50	<code>trajectory_eval.py:381 — median of per-case min(1.0, min_steps/actual)</code>
Step Efficiency mean	<code>trajectory_eval.py:382</code>

Metric	Formula location
Cost p50	<code>trajectory_eval.py:383</code>
Cost Max	<code>trajectory_eval.py:384</code>
Gap (outcome – trajectory)	<code>trajectory_eval.py:392</code>

Component	Location
Per-call judgement	<code>judge_call()</code> , <code>trajectory_eval.py:129</code>
Per-case judgement	<code>evaluate_case()</code> , <code>trajectory_eval.py:236</code>
Ordering rule	<code>_order_ok()</code> , <code>trajectory_eval.py:219</code>
Suite aggregation	<code>summarise()</code> , <code>trajectory_eval.py:356</code>
Corpus vocabulary	<code>corpus_vocabulary()</code> , <code>trajectory_eval.py:103</code>
Six failure modes	<code>FAILURE_MODES</code> , <code>trajectory_eval.py:53</code>
Outcome attachment	<code>score_outcomes()</code> , <code>trajectory_eval.py:406</code>
Self-test, 11 checks	<code>self_test()</code> , <code>trajectory_eval.py:580</code>
Root entry point	<code>trajectory_eval.py</code> (repo root), 20-line shim

The step-efficiency calculation itself sits inside `evaluate_case()` at `trajectory_eval.py:236`, computed per case before aggregation.

How to check it

```
python -m app.rag.trajectory_eval --self-test      # 11 checks
```

Notable checks: commutative order scores identically both ways; a two-hop wrong order fails; a question-supplied argument is not counted as hallucination; efficiency arithmetic is exactly $2/3 = 0.667$.

Task 3 — Outcome-vs-trajectory gap

What we did

Computed `Outcome Pass Rate – Trajectory Pass Rate` for both arms, and analysed one false-positive trace in detail.

Why we did it

The gap is the whole point of the week. If outcome and trajectory always agreed, trajectory evaluation would be redundant. Where they disagree tells you which one is lying.

Outcome and trajectory are computed by two independent code paths from the same run — `score_outcomes()` calls Week 7's *unmodified* `score_output`, while `evaluate_case()` reads only the tool records. That independence is what makes the gap meaningful rather than tautological.

What we found

arm	outcome	trajectory	gap
baseline	50.0%	80.0%	-30.0pp
mitigated	50.0%	90.0%	-40.0pp

The gap runs negative — the opposite of the textbook expectation. The trajectory is *easier* to pass than the answer. A flawed path is not what holds this agent back; three cases per run walk a clean path and fail on wording.

The false-positive trace

`report/w7/smoke_agent.json`, question `w7-10`, a real recorded run:

Query	"What is the rate limit on the /v1/embeddings endpoint?"
Tool calls	0
Answer	INSUFFICIENT_CONTEXT
Outcome	PASS
Trajectory	FAIL — 0 steps below <code>min_steps</code> 1

The answer is right. The agent never looked — it asserted the absence from pretraining rather than discovering it. `ANSWER_CONTRACT` rule 2 says "if the tools do not contain the answer", a claim about the tools that presupposes consulting them. The same reflex on a question the corpus **does** answer would confidently refuse retrievable material.

Where the code is

Thing	Location
FP / FN definition	<code>TrajectoryResult.false_positive</code> , <code>trajectory_eval.py:210</code> and <code>.false_negative:215</code>
Gap computation	<code>SuiteMetrics.gap_points</code> , <code>trajectory_eval.py:392</code>
Outcome attachment	<code>score_outcomes()</code> , <code>trajectory_eval.py:406</code>
Report artefact	<code>report/w8/gap_analysis.txt</code>
The FP trace itself	<code>report/w7/smoke_agent.json</code> , question <code>w7-10</code>

Task 4 — Exactly one mitigation

What we did

A **Consecutive-Search & Non-Existence Circuit Breaker**. One mitigation, one failure mode: tool thrashing.

Why this failure mode and not another

The Week 7 race recorded the agent on `w7-10` making 6 LLM calls and 5 consecutive `search_docs` calls before giving up — 20,680 tokens and 117,928 ms for a question whose correct answer is a one-call refusal.

Ranked against the other five modes, thrashing wins by a distance: **it is the only mode that produces an unbounded tail**. A wrong tool choice costs one call; thrashing costs as many as the budget allows. It owns Cost Max and Latency Max outright, which is why fixing it moves the maxima 63% and 87% while barely touching the medians.

Why it is not a prompt patch

The obvious fix — tell the model in the system prompt that repeated searches are futile — was rejected on a hard constraint:

Editing `ANSWER_CONTRACT` changes a string both Week 7 race arms depend on being byte-identical. The Week 7 fairness gate asserts exactly one difference between arms and aborts otherwise. The race would fail, and every committed Week 7 number would become incomparable.

So the fix sits in the **loop**, not the prompt. It wraps the contract's existing `chat` seam; the agent loop is unmodified.

Why the breaker never writes the answer

A vetoed call returns a **normal tool result** and the model still writes its own reply. A breaker that emitted the refusal itself would be scoring its own answer.

The three veto rules

Rule	Fires when	Why
Exact repeat	same <code>(tool, sorted-args)</code> seen before	Repeating a call cannot produce new information
Consecutive search limit	2 back-to-back searches return no chunk id not already seen	Retrieval has stopped producing evidence
Established absence	a tool already returned <code>found: false</code> for this target	Non-existence is settled

Rule 2 counts only **unproductive** searches — the streak resets whenever a search surfaces a new `chunk_id`. A search that finds new material is progress, however many preceded it.

Threshold justification: `max_consecutive_searches = 2` sits above every legitimate trajectory measured in Week 7 (deepest honest chain is 2 tool calls). It cannot fire on any correct trajectory in the suite — confirmed by the result: 9 of 10 cases untouched.

Where the code is

Thing	Location
Mitigation identifier	<code>MITIGATION_ID</code> , <code>week8_mitigation.py:61</code>
Thresholds	<code>BreakerConfig</code> , <code>week8_mitigation.py:64</code>
State machine	<code>BreakerState</code> , <code>week8_mitigation.py:79</code>
Learning from tool results	<code>observe_result()</code> , <code>week8_mitigation.py:87</code>
The three veto rules	<code>veto()</code> , <code>week8_mitigation.py:106-131</code>
— exact repeat	<code>week8_mitigation.py:112</code>
— consecutive search limit	<code>week8_mitigation.py:117</code>
— established absence	<code>week8_mitigation.py:125</code>
Vetoed-call payload	<code>breaker_payload()</code> , <code>week8_mitigation.py:137</code>
The seam wrapper	<code>wrap_chat()</code> , <code>week8_mitigation.py:150</code>
Entry point	<code>run_mitigated_agent()</code> , <code>week8_mitigation.py:214</code>
Self-test, 9 checks	<code>self_test()</code> , <code>week8_mitigation.py:235</code>

Measured before / after

	Before	After
Thrashing failures	1	0
Cost Max	\$0.003246	\$0.001193 (−63.2%)
Latency Max	127,041 ms	16,890 ms (−86.7%)
Tokens total	61,217	46,037 (−24.8%)
Outcome pass rate	50.0%	50.0% (unchanged)

Resource price paid: 1.19 ms CPU per LLM call (33.26 ms total across the run), 0 extra tokens, 0 extra calls, 1 veto issued.

Task 5 — Regression matrix, six modes

What we did

Scored both arms against all six named failure modes and proved no mode got worse.

Why we did it

A mitigation that fixes one failure by causing another is not a fix. The matrix is the evidence that the trade did not happen — and it has to cover all six modes, not just the targeted one, or it proves nothing about the other five.

Results

#	Failure mode	Before	After	Regression?
1	Redundant / thrashing tool calls	1	0	no (improved)
2	Argument hallucination / schema violation	0	0	no
3	Suboptimal step efficiency	1	0	no (improved)
4	Parametric guessing / bypassed verification	1	1	no
5	Premature refusal	0	0	no
6	Budget overrun	1	0	no (improved)

Regressions: 0.

Mode 4 is *unchanged, not improved*. The remaining instance is `w8-09`, where the agent searched before checking retirement. The breaker targets thrashing, not ordering, and deliberately does not touch it — one mitigation, one failure mode. **This is an open defect, reported as such.**

Only `w8-10` changed across all ten cases. Nine of ten are identical in steps, trajectory verdict and outcome verdict.

Where the code is

Thing	Location
Mode definitions	<code>FAILURE_MODES</code> , <code>trajectory_eval.py:53</code>
Mode assignment	<code>evaluate_case()</code> , <code>trajectory_eval.py:236</code>
Mode counting	<code>summarise()</code> , <code>trajectory_eval.py:356</code>
Before/after renderer	<code>render_compare()</code> , <code>trajectory_eval.py:492</code>
Report artefact	<code>report/w8/regression_matrix.md</code>

Week 3–7 regression evidence

All 14 checks pass; `week6_assertions --json` is byte-identical to the committed baseline; `git diff HEAD -- app/rag/week7_.py eval/week7_.yaml` is empty. **Week 8 is purely additive** — no Week 3–7 source file was modified.

Task 6 — Bonus defences

What we did

Four defences: indirect prompt injection detection, tool output sanitisation, read-only scope auditing, and generated-code guardrails.

Why we did it

The Week 7 agent feeds tool output straight back into the message list. That output is retrieved documentation — text the agent did not write and, in a real deployment, text a third party could edit. Anything instruction-shaped in a chunk arrives in exactly the position instructions arrive.

These defend against the shape of the pipeline, not a known attack. `docs/` is authored and clean; the self-test proves the defences against a synthetic injection corpus rather than an incident that has not happened.

Detection flags but never deletes. Dropping text would change what the agent can cite and make a retrieval bug indistinguishable from a defence firing.

Patterns are deliberately narrow. "Ignore" alone appears in legitimate documentation (`retry_backoff_ms` is "*accepted in v2 but ignored*"), so every pattern requires an instruction verb plus an object that only makes sense to an agent. Result: **0 false positives across all 38 real corpus chunks.**

Code guardrails scan fenced blocks only — prose mentioning "eval order is unchanged" is documentation, not a payload.

Where the code is

Defence	Location
Injection patterns (9)	<code>INJECTION_PATTERNS</code> , <code>week8_defense.py:63</code>
Injection scanner	<code>scan_injection()</code> , <code>week8_defense.py:109</code>
Code red flags (8)	<code>CODE_RED_FLAGS</code> , <code>week8_defense.py:78</code>
Code scanner	<code>scan_generated_code()</code> , <code>week8_defense.py:122</code>
Data fence constants	<code>week8_defense.py:89-90</code>
Sanitiser	<code>sanitise_tool_output()</code> , <code>week8_defense.py:136</code>
Write-capable call list	<code>WRITE_CALLS</code> , <code>week8_defense.py:159</code>
AST scope audit	<code>audit_read_only()</code> , <code>week8_defense.py:165</code>
Overhead measurement	<code>measure_overhead()</code> , <code>week8_defense.py:257</code>
Self-test, 10 checks	<code>self_test()</code> , <code>week8_defense.py:198</code>

Measured overhead

Sanitisation: **0.40 ms per call** on a real 4,205-char `search_docs` payload — 0.004% of a ~9,600 ms median query.

Scope audit result: all three tools and `dispatch` have **zero** write-capable calls.

Task 7 — Machine telemetry

What we did

`report/w8/trajectory_eval.json`, 30 KB, three arms: baseline agent, mitigated agent, fixed workflow.

Why we did it

The numbers in every report must be regenerable from data, not retyped. The JSON carries the exact command that produced it and the source path of every arm.

The fixed workflow arm carries outcome and cost only, with an explicit note saying why: its path is fixed at import time (4 dispatch + 1 chat call sites, statically proved in Week 7), so tool-choice accuracy and step efficiency are not meaningful for it. Including them would invent a comparison that does not exist.

Where the code is

Thing	Location
JSON writer	<code>main()</code> --json branch, <code>trajectory_eval.py:681</code>
Metrics model	<code>SuiteMetrics</code> , <code>trajectory_eval.py:326</code>
Per-case model	<code>TrajectoryResult</code> , <code>trajectory_eval.py:187</code>
Artefact	<code>report/w8/trajectory_eval.json</code>

Three arms compared

Arm	Outcome	Cost Max	Tokens total
Baseline agent	50.0%	\$0.003246	61,217
Mitigated agent	50.0%	\$0.001193	46,037
Fixed workflow	50.0%	\$0.000670	29,720

The mitigation closes roughly half the agent-to-workflow gap on worst-case cost without changing what either answers. It does **not** make the agent cheaper than the workflow — Week 7's FIXED WORKFLOW verdict stands.

Complete file map

File	Lines	Role
<code>eval/week8_cases.yaml</code>	278	Trajectory case spec, 10 cases
<code>app/rag/trajectory_eval.py</code>	750	Metrics engine, gap, regression comparison
<code>trajectory_eval.py</code>	20	Repo-root entry point (shim)
<code>app/rag/week8_mitigation.py</code>	374	The one mitigation
<code>app/rag/week8_defense.py</code>	322	Four bonus defences
<code>report/w8/gap_analysis.txt</code>	—	Task 3 artefact
<code>report/w8/regression_matrix.md</code>	—	Task 5 artefact
<code>report/w8/trajectory_eval.json</code>	30 KB	Task 7 artefact
<code>report/w8/baseline_runs.json</code>	143 KB	Baseline capture (input)
<code>report/w8/mitigated_runs.json</code>	128 KB	Mitigated capture (input)
<code>report/w8/build_week8_pdf.py</code>	220	Markdown to PDF renderer

How to verify everything, offline

No API key, no tokens, no network:

```
python -m app.rag.trajectory_eval --validate      # spec vs corpus
python -m app.rag.trajectory_eval --self-test    # 11 engine checks
python -m app.rag.week8_mitigation --self-test   # 9 breaker checks
python -m app.rag.week8_defense --self-test     # 10 defence checks
python -m app.rag.week8_defense --audit         # read-only scope audit
python -m app.rag.week8_defense --overhead      # sanitisation cost

# regenerate every headline number from saved runs
python -m app.rag.trajectory_eval \
    --replay report/w8/baseline_runs.json --label baseline \
    --compare report/w8/mitigated_runs.json \
    --json report/w8/trajectory_eval.json
```

Only two commands ever cost tokens, and both outputs are already committed:

```
python -m app.rag.week7_agent --all --delay 12 \
    --out report/w8/baseline_runs.json          # ~61k tokens
python -m app.rag.week8_mitigation --run --delay 12 \
    --out report/w8/mitigated_runs.json         # ~46k tokens
```

What a reviewer should challenge

The spec change made after seeing the data. Across 18 agent runs there were zero false positives. Re-reading the spec, [w8-10](#) carried `min_steps: 0`, which made failure mode 4 unmeasurable by construction on the only case where it can occur. It was changed to 1 and a real recorded run surfaced as an FP.

The justification is that `ANSWER_CONTRACT` rule 2 presupposes consulting the tools, so a zero-call refusal does not satisfy it — and the change is documented in the case's own `rationale` field at `eval/week8_cases.yaml`, not only in this report. But it was made after the data came back empty, and a reviewer is entitled to discount it on those grounds. The underlying *phenomenon* — a zero-call refusal on [w7-10](#) — is real and recorded regardless of how the spec scores it.

Tool-Choice Accuracy is 100% on both arms, which reflects `allowed_tools` being deliberately permissive rather than the agent being flawless. This metric has little headroom on this question set.

Both captures are reps=1. The agent is non-deterministic at temperature 0 and [w7-10](#) is demonstrably bistable. Aggregate direction is robust; individual cells should not be over-read.

Mode 4 remains open at [w8-09](#). Not fixed, by design — the brief permits one mitigation.